5C37 Assignment 1

Dr. David Corrigan

February 6, 2026

1 Introduction

This assignment is worth 20% of your final mark. The deadline for submission of your NUKE script and presentation is as shown in Blackboard. PLEASE USE THE FILENAME CONVENTION LastName_FirstName.XXX when submitting your NUKE graph. You will be shown what data we will be using and you do not need to upload that data.

2 The Task

In lectures, you worked through several levels of sophistication in designing an algorithm for image keying with colour. For this assignment, you must design and implement an algorithm for binary colour keying using the data provided in lectures. Your algorithm should possess the following characteristics.

- It should implement a Bayesian interpretation of the problem for video.
- Use a Gaussian distribution for the background colour distribution. The parameters for that Gaussian distribution should be estimated from a block of pixels on a frame in the background.
- Use a Spatial MRF as a prior for the segmentation label field. Use a neighbourhood of 3×3 pixels (8-pixel neighbourhood).
- Use the iterative algorithm outlined in Section 5 to get the segmentation mask. Use synchronous updating (i.e. update all pixels concurrently in a single iteration).
- Expose key algorithm parameters to the node properties panel so that they can be varied without having to recompile the blink nodes.

3 The Implementation

You must implement your algorithm using NUKE scripting and node graphs. Remember that you might need to repeat nodes to achieve certain effects e.g. iterating over the same image. Use a combination of NUKE nodes and blink scripts.

4 Deliverables & Assessment

Assessment will be based on a 10 minute presentation and code demonstration followed by a 10 minute Q&A session as well as a review of your submitted materials.

The implementation must run on 5 frames input material at HD resolution. The following should be submitted for marking.

- The NUKE node graph implementing your solution.
- You will have to demonstrate your implementation running correctly to the demonstrator. This will also require you to present and explain your work coherently.
- Choose and implement a range of metrics (at least 2) to assess the quality of your results.

- A Powerpoint presentation (about 10 slides) that describes your implementation. The presentation should contain
  1. A mathematical description of the algorithm, including equations that describe the likelihood and prior you used.
  2. A description of how your algorithm maps onto your implementation.
  3. An assessment of the accuracy/fidelity of your extracted mask and composite image output. Describe the assessment criteria you use. At least 2 metrics should be used to compare results.
  4. A comparison with the U-Net estimator described in Section 6 and a built-in Nuke keyer.
  5. A description of shortcomings of your algorithm and suggestions for improvement.

4.1 Marking Scheme

The assignment is marked out of 100.

- The Nuke script works, and fulfills all the specifications of the assignment. (30 marks)
- Mathematic and implementation descriptions are correct, and are consistent with each other and the submitted nuke graph. (20 marks)
- Experiments: Chosen experiments are sensible. Quality assessment metrics chosen appropriately. Results are sensible and are illustrated properly using plots and images are properly formatted. Proper conclusions and improvement suggestions drawn (30 marks)
- The student is able to present their work and respond cogently to questions (20 marks)

5 The Segmentation Algorithm

The algorithm you should use for the assignment should be based on the Bayesian Keyer/Matter (MAP) covered in Handout 2 of the course notes.

Recall that in this algorithm we seek to find the matte $\hat{\boldsymbol{\alpha}}$ that satisfies

$$\hat{\boldsymbol{\alpha}} = \arg\max_{\boldsymbol{\alpha}} P(\boldsymbol{\alpha}|\mathbf{c}), \tag{1}$$

where $\mathbf{c}$ is the image we want to matte. Using Bayes law and ignoring the evidence distribution we can factorise the posterior into a likelihood and prior, getting

$$\hat{\boldsymbol{\alpha}} = \arg\max_{\boldsymbol{\alpha}} P(\mathbf{c}|\boldsymbol{\alpha})P(\boldsymbol{\alpha}). \tag{2}$$

Suitable expressions for the likelihood and prior are given in the handout. Choose the expressions you feel are most appropriate for the problem and fit the requirements of the assignment.

We can convert this MAP problem into an energy minimisation problem by computing negative logs of the likelihood and prior as follows

$$\hat{\boldsymbol{\alpha}} = \arg\min_{\boldsymbol{\alpha}} \left(-\ln\left(P(\mathbf{c}|\boldsymbol{\alpha})\right) - \ln\left(P(\boldsymbol{\alpha})\right)\right). \tag{3}$$

5.1 Optimising the Posterior

To compute the minimum energy solution, we will use an optimisation algorithm similar to the Iterated Condtional Modes (ICM) algorithm that independently updates individual pixels rather than jointly optimise over all pixels. It is also an iterative optimisation algorithm, where you iterate until successive iterations of the matte are the same up to a maximum limit of iterations.

For each pixel $\boldsymbol{p}$ we wish to find the matte value at iteration t, $\alpha_t[\boldsymbol{p}]$. We assume that we know the value of the matte at the previous iteration $\boldsymbol{\alpha}_{t-1}$. We can write an expression for updating the value of a single pixel as follows

$$\alpha_t[\mathbf{p}] = \arg\min_{\alpha[\mathbf{p}]} \left(-\ln\left(P(c[\mathbf{p}]|\alpha_t[\mathbf{p}])\right) - \ln\left(P(\alpha_t[\mathbf{p}])\right)\right). \tag{4}$$

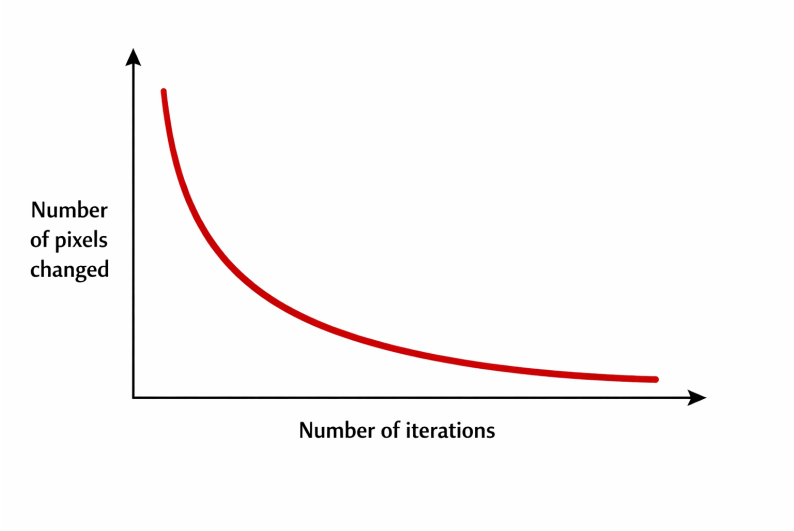

Figure 1 : Typically in an iterative algorithm like this, a large number of pixels change in the first iteration. After a small number of iterations the number of pixels that change will be much lower. Convergence is usually reached within 10 iterations.

In the keying algorithm we looked at in Handout 2, we looked at likelihood and prior expressions that assume conditional independence between pixels. We can use them to derive the likelihood and prior expressions for this expression.

Since there are only 2 values for $\alpha_t[\mathbf{p}]$ (0 and 1), defining the rule to determine the minimum is straightforward:

$$\alpha_t[\mathbf{p}] = \begin{cases} 1 & \text{if } (-\ln(P(c[\mathbf{p}]|\alpha_t[\mathbf{p}]=1)) - \ln(P(\alpha_t[\mathbf{p}]=1))) \\ & \quad < (-\ln(P(c[\mathbf{p}]|\alpha_t[\mathbf{p}]=0)) - \ln(P(\alpha_t[\mathbf{p}]=0))) \\ 0 & \text{otherwise.} \end{cases} \tag{5}$$

We are simply computing the sum of the likelihood and priors for both $\alpha_t[\mathbf{p}] = 0$ and $\alpha_t[\mathbf{p}] = 1$ and chosing the value with the lower total.

5.2 Dealing with the Prior

Recall that in Handout 2 we used a spatial smoothness prior based on a Markov Random Field (MRF). This was an expression of the form $P(\alpha[\mathbf{p}]|\mathcal{N}(\alpha[\mathbf{p}]))$ where $\mathcal{N}(\alpha[\mathbf{p}])$ is the set of α values of pixels that are within the spatial neighbourhood of $\mathbf{p}$. A general expression of the negative log prior (i.e. the prior energy) is as follows

$$-\ln(P(\alpha[\mathbf{p}]|\mathcal{N}(\alpha[\mathbf{p}]))) = \Lambda \sum_{\mathbf{q}\in\mathcal{N}[\mathbf{p}]} \lambda_{\mathbf{p},\mathbf{q}} f(\alpha[\mathbf{p}], \alpha[\mathbf{q}]). \tag{6}$$

In this equation Λ is a parameter that confirms the relative strength of the prior and likelihood terms in the total energy and $\lambda_{\mathbf{p},\mathbf{q}}$ is a local weight for each pair of pixels $\mathbf{p}$ and $\mathbf{q}$. $f(\alpha[\mathbf{p}], \alpha[\mathbf{q}])$ is a function that defines the cost of assigning the values $\alpha[\mathbf{p}]$ and $\alpha[\mathbf{q}]$ to $\mathbf{p}$ and $\mathbf{q}$. It is up to you to determine suitable values for these quantities in this assignment.

A Problem

The prior expression above requires us to know the value of α at both positions $\mathbf{p}$ and $\mathbf{q}$ to compute the prior energy at iteration t. However, since we want to take advantage of the GPU architecture by updating all pixels pixels concurrently, we don't have knowledge of α at **any** other pixel $\mathbf{q}$ for the current iteration. To overcome this, we can use the values from the previous iteration $t-1$. In effect, we compute

$$-\ln(P(\alpha_t[\mathbf{p}]|\mathcal{N}(\alpha_{t-1}[\mathbf{p}]))) = \Lambda \sum_{\mathbf{q}\in\mathcal{N}[\mathbf{p}]} \lambda_{\mathbf{p},\mathbf{q}} f(\alpha_t[\mathbf{p}], \alpha_{t-1}[\mathbf{q}]) \tag{7}$$

instead.

5.3 Implementing this in Nuke

Create a Blink node that computes exactly one iteration of the optimisation. As a suggestion this node should have at least 2 inputs: the image to be segmented and the matte from the previous iteration. The examples covered in the introduction to Nuke lab should give you enough knowledge of Nuke and Blinkscript to achieve this. Note that we also covered an example that showed how to chain multiple copies of the same Blink node to iteratively process an image.

Note that you have to decide how to set the initial value of the matte, and its choice has a major impact on the performance of the algorithm. You will have to compute the initial value yourself and cannot use any of the supplied materials or another keyer that you havent coded.

6 A Segmentation U-Net

Consider a neural network trained for object matting. While such a network might produce acceptable results, it typically falls short of the excellence required for professional post-production.

To address this, one strategy is to deliberately overfit a neural network on a labeled subset of a frame sequence. Our priority here is to ensure that the neural network performs almost perfectly on our specific sequence of frames. By overfitting to our data, we can compel the neural network to excel in processing our frame sequence. Although we still need to generate a small training dataset from our sequence (e.g. manually labelling/matting 2 frames out of a frame sequence), this approach is significantly less labor-intensive compared to manually matting all frames in the sequence.

To ensure that our matted objects in the selected subset of frames accurately represent the region across the entire frame sequence, we must employ data augmentation techniques. As objects move through a scene, their lighting, orientation, and position change. Our data augmentation steps should include transformations that mimic typical changes experienced by an object in a scene, thereby enhancing the neural network's ability to generalize from our limited training dataset to the entire sequence.

By incorporating limited manual data labelling/matting over a subset of a frame sequence and robust data augmentation algorithms, we should be able to train a neural that yields excellent performance with minimal effort for that specific frame sequence.

In this assignment, we have provided you with a U-Net trained with data augmentation using blurring, rotations and gamma shifting.

Objective

You are required to:

1. Convert the PyTorch Model File to a Cattery File (Using the *CatFileCreator* Node]).
2. Perform Inference and write out your mattes to file.
3. Include the results obtained from using the NN as a comparison point when presenting your work.

Nuke Hints

The node you have to use in Nuke to complete this section is the *CatFileCreator* node. A description of the settings can be found in the Foundry's reference. You can follow along with the Foundry's reference for creating Cat Files from Pytorch models here.

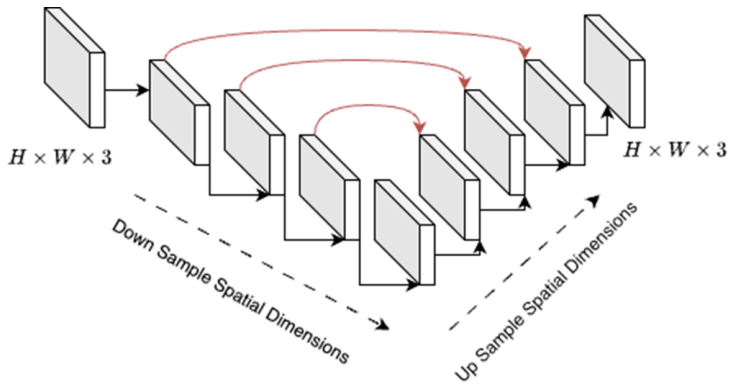

Figure 2 : Architecture Diagram of the U-Net. It consists of 4 downsampling and 4 upsampling blocks. Downsampling/upsampling are done by a factor of 2. It has a trainable parameter count of approximately 525k. The final activation function is a Sigmoid function. This ensures that the output lies within 0 and 1.